Architecture globale — EFE + Scout & Synthesize

Document de référence unifié. Décrit l'intégralité du système tel qu'il existe (EFE = plateforme nano-agent audit-grade) et tel qu'il s'étend (S&S = application "recherche documentaire" qui consomme la plateforme). Rédigé pour être *implementation-grade* : un ingénieur entrant doit pouvoir, à partir de ce seul document, comprendre la topologie complète, identifier le composant pertinent à une question donnée, et lire ensuite le code en sachant où il atterrit. **Version** : 1.0 — 23 mai 2026 **Statut** : architecture vivante. Les sections marquées [SHIPPED] décrivent du code *shippé et testé*. Les sections marquées [STUB] décrivent du code *stubbé mais avec interfaces stables*. Les sections marquées [S&S] décrivent l'extension *Scout & Synthesize* qui consomme la plateforme.

Table des matières

1 Abstract
2 Vision, objectifs, principes, non-objectifs
3 Vue système globale
4 Topologie de processus et de nœuds
5 Couche 1 — Inférence latent-space
6 Couche 2 — Meta-Swarm A1 (fédération)
7 Couche 3 — Audit transverse (HMAC chains + Merkle)
8 Couche 4 — Nano Agent Platform (le cœur)

1. SDK et types canoniques 2. Catalog 3. Modes d'exécution A / B / C 4. Dispatcher et DAG 5. Blackboard et stigmergie 6. Swarm Orchestrator + SwarmAgent 7. Scheduler de fairness 8. Idempotency + cache 9. Provenance LLM 10. Publisher Identity + KYC 11. Gold Test Gate 12. Multi-Region Replay 13. Agentic Bridge (Claude) 14. Gateway HTTP + MCP Server

1 Application Scout & Synthesize
2 Architecture des données
3 Plan de contrôle et streaming
4 Plan d'inférence
5 Sécurité, isolation, conformité
6 Observabilité
7 Tenancy et configuration
8 Évaluation et boucles qualité
9 Cycle de vie d'une requête (E2E)
10 Index des crates (47 crates)
11 Glossaire

0. Abstract

Le système combine **deux artefacts conceptuels distincts** intégrés dans un seul exécutable :

- **EFE** (*Ephemeral Fluid Swarm*) — une plateforme Rust de 47 crates qui transforme les flux d'agents IA en opérations cryptographiquement auditables. Quatre couches : (1) inférence latent-space, (2) coordination meta-swarm fédérée, (3) audit transverse HMAC+Merkle, (4) plateforme nano-agent multi-mode.

- **Scout & Synthesize** (*S&S*) — une application bâtie sur EFE qui exécute un DAG préconçu de nano-agents pour répondre à des questions documentaires avec latence perçue < 250 ms, fact-checking parallèle, et streaming SSE. Elle réutilise *intégralement* la plateforme EFE en n'ajoutant que la logique métier spécifique à la recherche.

Les **patrons d'intelligence en essaim** (phéromones SwarmSys, beacon protocol Symphony, drafter/verifier Speculative RAG) sont *intégrés au blackboard EFE* via une extension non destructive : champ `pheromone: f32`, décroissance exponentielle λ=1/24h, K-means clustering pour la diversité forcée des scouts.

L'architecture est conçue pour deux marchés simultanés :

- **B2B régulé** (pharma, finance, défense) — moat = audit cryptographique non-retrofittable
- **B2A développeur** (agents qui appellent EFE pour leurs sous-tâches) — moat = vitesse + déterminisme + écosystème publisher

---

1. Vision, objectifs, principes, non-objectifs

1.1 Vision

> Remplacer la cascade séquentielle d'appels LLM par un essaim parallèle de nano-agents spécialisés et auditables, chacun produisant une sortie schema-validée et une trace cryptographique réplayable sur sept ans, exposés à la fois aux entreprises régulées (via MCP server) et à l'utilisateur final (via gateway HTTP+SSE).

1.2 Objectifs mesurables

Domaine	Objectif	Métrique
Latence perçue (S&S)	< 250 ms `time_to_first_useful_pixel`	UI SSE event `source`
Latence réponse (S&S)	< 1 s `time_to_first_token` p50	UI SSE event `token`
Coût LLM (vs naïf)	≤ 30 % d'un appel monolithique	Provenance per-call
Déterminisme Mode A	100 % sur 1 M de runs	`DeterminismChecker`
Déterminisme Mode B	99.9 % avec seed fixé	`DeterminismChecker`
Déterminisme Mode C	≥ 95 % sur fenêtre de cache	`ReplayEngine`
Audit retention	7 ans (tier audit), 7 j (free), avec proof Merkle en O(log n)	HMAC chain + Merkle epoch
Throughput Mode A	≥ 10 K req/s/node	bench harness
Throughput Mode B	≥ 1 K req/s/node	bench harness
Throughput Mode C	≥ 100 req/s/node	bench harness
Disponibilité	99.9 % single-region, 99.99 % multi-region	SLO rollup

1.3 Principes architecturaux

1. **Audit par construction** — *aucune* opération ne contourne le HMAC chain. Pas d'opt-out.
2. **Schema-validated I/O** — chaque agent a un JSON schema d'entrée et de sortie figé par version.
3. **Determinism tiers** — Mode A = mathématique, Mode B = seeded, Mode C = cache-replayable. Aucune opération n'est "indéterministe".
4. **Stigmergic coordination** — les agents communiquent via le blackboard, pas directement entre eux.

5 **Tier-based trust** — un agent publisher monte par paliers (0/1/2/audit) avec garanties contractuelles croissantes.

6 **Tenant isolation by default** — clés HMAC, quotas, chaînes d'audit *toutes* par tenant. Pas d'état partagé.

7 **Hybrid edge + cloud** — les nanos tournent localement (edge ou serveur privé) ; le gros LLM est appelé via API avec prompt caching.

8 **Streaming-first UX** — chaque couche émet des événements SSE pour découpler la latence ressentie de la latence totale.

9 **Stable interfaces, mutable implementations** — les traits Rust et les `*.proto` sont versionnés et stables sur 12+ mois ; le code derrière peut être réécrit sans casser l'API.

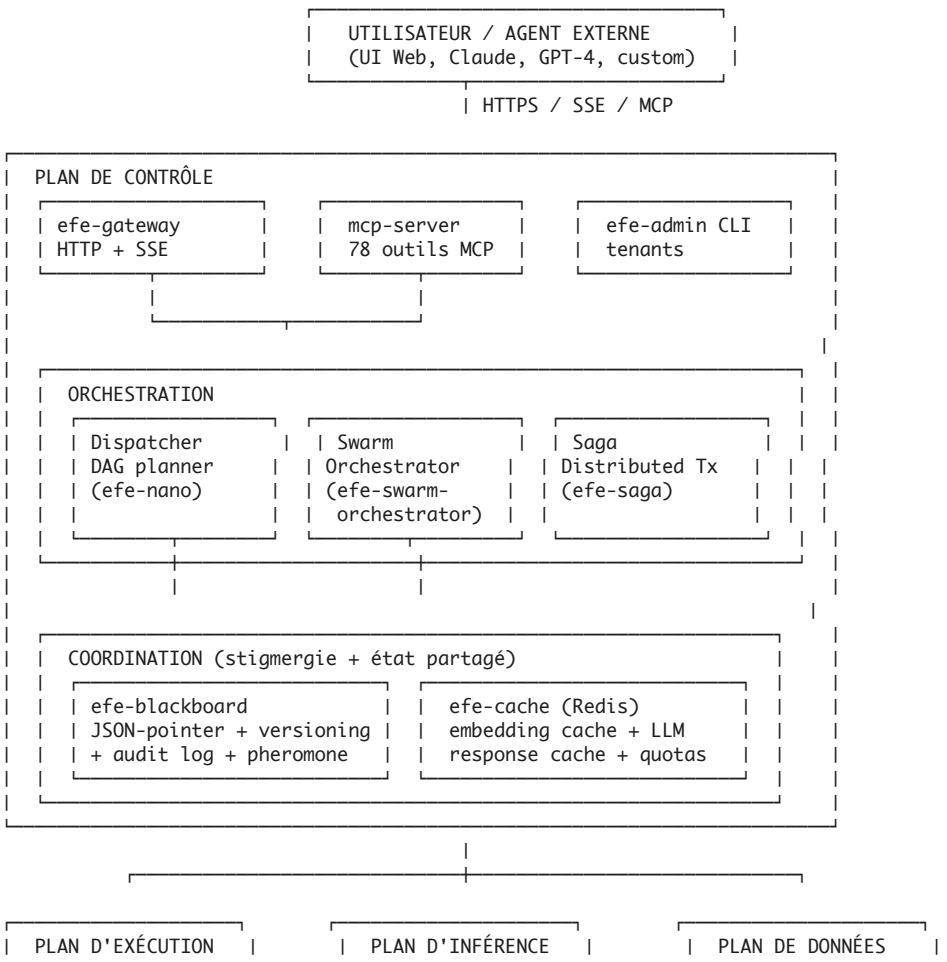
10 **Additive strict refactor** — aucun crate existant n'est modifié pour ajouter une nouvelle capacité ; on ajoute un nouveau crate.

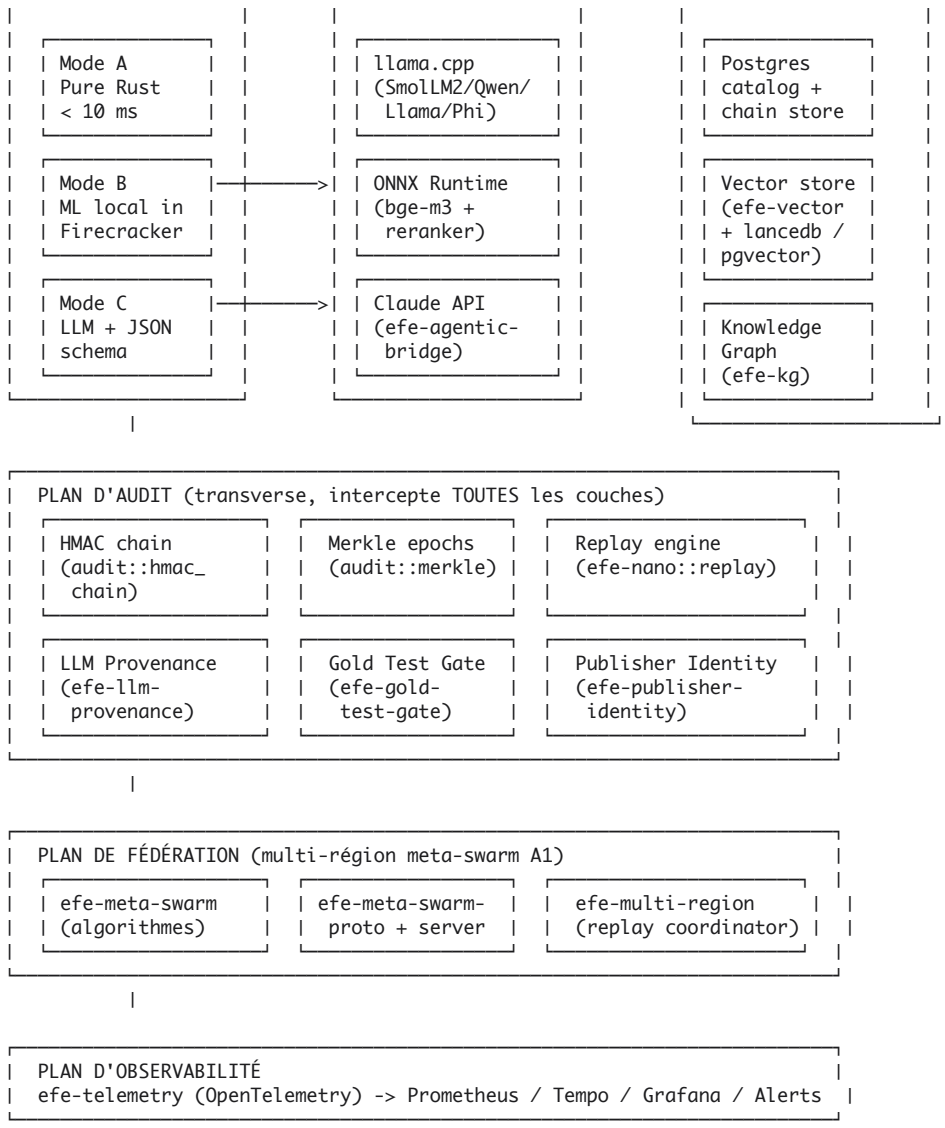
1.4 Non-objectifs (explicites)

- [N/A] Construire un LLM généraliste — on consomme Claude/GPT/Gemini, on ne les remplace pas.
- [N/A] Multi-tour conversationnel complexe — V1 = QA atomique, V2 envisagée pour conversation.
- [N/A] Génération créative — l'audit-grade implique structured output, pas de free-form.
- [N/A] Inference Apple Metal optimisée pour le grand public — focus serveur Linux + edge Mac M-series pour devs.
- [N/A] Couche d'autonomie globale (AGI-style boucle) — chaque agent fait *une* sous-tâche puis s'arrête.

---

2. Vue système globale





Sept plans logiques :

- **Contrôle** — entrée des requêtes (gateway, MCP, admin)
- **Orchestration** — décide qui exécute quoi (Dispatcher, Swarm Orchestrator, Saga)
- **Coordination** — état partagé entre agents (Blackboard, Cache)
- **Exécution** — où s'exécutent les nano-agents (Mode A/B/C)
- **Inférence** — où vivent les modèles (llama.cpp, ONNX, Claude API)
- **Données** — persistance (Postgres, vector store, KG)
- **Audit** — traçabilité (HMAC, Merkle, Provenance, Gold Gate, Publisher Identity)
- **Fédération** — multi-région
- **Observabilité** — tracing/metrics

Ces plans sont *orthogonaux* : on peut déployer un seul tenant sans fédération, ou un seul mode (A) sans inférence externe.

---

3. Topologie de processus et de nœuds

3.1 Processus par nœud (single-tenant deployment)

Process	Binaire	Rôle	Mémoire	Latence cible
`efe-server`	`efe-server` (Rust)	gRPC inference engine + dispatcher	4–8 Go	< 10 ms
`efe-gateway`	`efe-gateway` (Rust)	HTTP + SSE entry point	1 Go	< 5 ms
`mcp-server`	`mcp-server.py`	MCP protocol, 78 outils	1 Go	< 20 ms
`efe-nano-inference-daemon`	musl statique	guest-side daemon dans Firecracker	200 Mo / VM	varie selon mode
`efe-meta-swarm-server`	`efe-meta-swarm-server`	controller + participant gRPC	2 Go	< 50 ms inter-région
`postgres`	postgresql 16	catalog, chain store, metering	2–4 Go	< 5 ms
`redis`	redis 7	cache embeddings + LLM responses + quotas	1–2 Go	< 1 ms
`nats-jetstream`	nats-server	events typed pub/sub (efe-events)	512 Mo	< 5 ms
`clickhouse` (optionnel)	clickhouse-server	aggregation metering haute volumétrie	4 Go	scale-driven
`llama-server` (×N)	llama.cpp	un par modèle SLM (SmolLM2, Qwen, Llama, Phi)	250 Mo–2.4 Go	< 100 ms
Firecracker VM pool	KVM	sandbox Mode B/C, 1 VM par invocation	256 Mo / VM	200 ms boot
Tempo + Prometheus + Grafana	observability stack	tracing + metrics	2 Go	—

3.2 Topologies de déploiement

Mode développeur (1 machine, Mac M-series ou Linux laptop) :

- Tous les process tournent localement
- Firecracker remplacé par `StubVmPool` (Mode B/C tournent in-process)
- Postgres et Redis dans Docker Compose
- Pas de fédération multi-région

Mode single-tenant production (1 cluster Kubernetes) :

- `efe-server` × 3 réplicas avec affinity
- `efe-gateway` × 2 réplicas derrière un LB
- Firecracker pool sur worker nodes KVM-enabled
- Postgres en HA avec read replicas
- NATS JetStream en cluster 3 nœuds
- Redis Sentinel
- Observability stack dédié

Mode multi-tenant régulé (SaaS, hosted) :

- Comme single-tenant *par tenant* (logical isolation forte)
- Plus un control plane partagé (admin, billing, metering)
- Clés HMAC distinctes par tenant (cf. §6)
- Tenant-affinity scheduling pour réduire le bruit cross-tenant

Mode federated multi-région (audit-grade) :

- N déploiements single-tenant complets + connectés via `efe-meta-swarm-server`
- Données *jamais* répliquées ; seuls les signaux de consensus de haut niveau circulent
- Replay possible depuis n'importe quelle région via `efe-multi-region`

---

4. Couche 1 — Inférence latent-space

[SHIPPED]

4.1 Concept

L'inférence latent-space est le cœur technique différentiel d'EFE. Elle exécute des *swarms d'agents mathématiques* dans un espace vectoriel partagé. Chaque agent applique une transformation tensorielle (un noyau CUDA), et l'essaim converge vers un point de consensus en quelques itérations.

Différence avec un appel LLM single-shot :

- Un LLM produit *un* point de prédiction.
- Un essaim latent-space produit une *distribution de consensus* avec une mesure d'incertitude calibrée.
- Le coût marginal d'un agent additionnel est ~µs (kernel CUDA fusionné), vs ~secondes pour un LLM.

4.2 Crates impliqués

Crate	Rôle	LOC approx.
`efe-core`	Types, erreurs, orchestration interne	8 K
`efe-kernels`	Bindings CUDA + kernels fusionnés	4 K
`efe-ffi`	Bindings PyO3 (Python ↔ Rust)	1.5 K
`efe-server`	gRPC public surface (M10)	3 K
`efe-telemetry`	Tracing + metrics tracing-rs	1 K

4.3 API publique (extrait)

```
// efe-core
pub trait LatentSwarm {
    fn iterate(&mut self, steps: usize) -> SwarmState;
    fn consensus(&self) -> ConsensusResult;
    fn safety_invariants(&self) -> SafetyReport;
}

pub struct ConsensusResult {
    pub centroid: Tensor,
    pub variance: f32,
    pub member_states: Vec<AgentState>,
    pub iterations_to_converge: u32,
    pub safety_ok: bool,
}
```

4.4 Garanties

- **Convergence prouvée** sous contraintes de norme (cf. `efe-meta-swarm/src/certification.rs`)
- **Safety hierarchical** : les invariants des swarms enfants sont garantis par induction sur les parents
- **Reproducibilité par snapshot** : `SwarmState` sérialise tout l'état nécessaire pour rejouer la convergence

---

5. Couche 2 — Meta-Swarm A1 (fédération)

[SHIPPED]

5.1 Concept

A1 (*Aggregation 1*) est la couche qui permet à plusieurs instances EFE indépendantes de collaborer *sans partager leurs états internes*. Une instance pharmaceutique européenne et une instance pharmaceutique américaine peuvent échanger des *signaux de consensus* sans qu'aucune donnée brute ne traverse l'Atlantique.

5.2 Crates impliqués

Crate	Rôle
`efe-meta-swarm`	Algorithmes core (in-process)
`efe-meta-swarm-proto`	Définitions Protocol Buffers + tonic stubs
`efe-meta-swarm-server`	Binaires Tonic : `controller` + `participant`
`efe-multi-region`	Coordinateur de replay multi-région

5.3 Protocole

Chaque instance EFE est soit un **controller** (init + final synthesis), soit un **participant** (compute local + return consensus signal). Le `controller` :

1. Broadcast un `MetaTask { task_id, query, schema }` à N participants
2. Reçoit des `ParticipantConsensus { task_id, centroid, confidence, region }`
3. Calcule un `MetaConsensus` (weighted vote + outlier detection)
4. Retourne uniquement le `MetaConsensus` au caller

Aucune donnée brute ne traverse la frontière. Seuls les centroids vectoriels (haute dimension mais sans information sémantique reconstructible) + scores de confiance.

5.4 Activation à 4 niveaux

L'A1 est *opt-in* à quatre niveaux pour la rétrocompatibilité :

1. **Cluster** : `EFE_META_SWARM_ENABLED=true`
2. **Tenant** : `tenant.meta_swarm.enabled = true` dans Postgres
3. **Request** : header `X-Meta-Swarm: true`
4. **Session** : flag sur le `SwarmRequest`

En Phase 1 (mai-juillet 2026), le binaire `efe-server` est *byte-identique* à la version pré-A1 si aucun de ces flags n'est activé.

---

6. Couche 3 — Audit transverse

[SHIPPED]

6.1 Principe

Aucune opération du système n'est exécutée sans produire une ligne d'audit. Cette propriété est garantie *par le type system Rust* : tout `Runtime` implémentant `NanoAgentExecutor` reçoit un `CallContext` contenant un `Arc<dyn`

`AuditWriter>`, et chaque appel d'agent émet exactement une `AuditEntry` via cet `AuditWriter`.

6.2 Structure d'une `AuditEntry`

```
pub struct AuditEntry {
    // Identification
    pub entry_id: Uuid,                  // unique per entry
    pub tenant_id: TenantId,             // tenant isolation
    pub request_id: Uuid,                // groups entries of a swarm
    pub op_id: String,                   // operation within the swarm
    pub agent_ref: NanoAgentRef,         // which agent ran
    pub run_mode: RunMode,               // A / B / C

    // Temporal
    pub started_at: SystemTime,
    pub finished_at: SystemTime,
    pub duration_ms: u64,

    // Payload (hashed or full per policy)
    pub input_hash: [u8; 32],            // BLAKE3 of canonicalized input
    pub output_hash: [u8; 32],
    pub full_input: Option<serde_json::Value>,   // gated by retention tier
    pub full_output: Option<serde_json::Value>,

    // Status
    pub status: NanoOpStatus,            // Success / Failed / Timeout / Cancelled
    pub error: Option<NanoErrorBody>,

    // Cryptographic chain
    pub prev_hash: [u8; 32],             // hash of previous entry (HMAC chain)
    pub entry_hmac: [u8; 32],            // HMAC(entry_id || prev_hash || payload, tenant_key)
    pub chain_seq: u64,                  // monotonic per tenant

    // Provenance (LLM only, Mode C)
    pub llm_provenance: Option<LlmProvenanceRecord>,

    // Tags + extension
    pub tags: Vec<String>,
    pub blackboard_writes: Vec<BlackboardWriteRef>,
}
```

6.3 Garanties cryptographiques

- **HMAC chain par tenant** : chaque entrée intègre le hash de la précédente, formant un linked list cryptographique. Toute modification rétroactive est détectable.
- **Clé HMAC par tenant** dérivée via HKDF d'une racine en KMS (Vault, AWS KMS, GCP KMS).
- **Merkle tree périodique** (epoch = 1 heure par défaut) : la racine du Merkle est ancrée dans `audit::merkle::EpochRoot` et peut être signée par cosign pour les tiers audit.
- **Inclusion proof en O(log n)** même sur des chaînes à 10■ entrées : un régulateur reçoit un proof court, vérifiable hors-ligne.
- **GDPR tombstone** : on peut supprimer le `full_input/full_output` d'une entrée sans rompre la chaîne (le `input_hash/output_hash` reste, et un `tombstone_reason` est ajouté).

6.4 Crates impliqués

Crate / module	Rôle
`efe-nano::audit::hmac_chain`	HMAC chain writer
`efe-nano::audit::chain_store`	Trait + InMemory + (futur) Postgres
`efe-nano::audit::hkdf`	Key derivation (env / static / Vault)
`efe-nano::audit::verify`	Vérification de la chaîne + sous-chaîne
`efe-nano::audit::merkle`	Merkle epoch roots + inclusion proofs
`efe-nano::replay`	Replay engine + DeterminismChecker

6.5 Tiers de rétention

Tier	Rétention payload	Rétention hash	Cosign	Cible
Free	7 jours	30 jours	non	Devs onboarding
Developer	30 jours	1 an	non	Devs pro
Professional	90 jours	1 an	non	Startups
Enterprise	7 ans	7 ans	non	Grands comptes
Audit-grade	7+ ans	7+ ans	**oui** (Sigstore Fulcio + Rekor)	Pharma, finance, défense

---

7. Couche 4 — Nano Agent Platform

[SHIPPED]

C'est le cœur produit. Sous-couche par sous-couche :

7.1 SDK et types canoniques

[SHIPPED] — Crate : `efe-nano-sdk` (MIT, public).

Définit *tous* les types qui traversent les frontières publiques :

```
pub struct NanoAgentId(String);          // canonical kebab-case
pub struct NanoAgentRef { id: NanoAgentId, version: Option<SemVer> }
pub struct TenantId(Uuid);
pub struct SwarmId(Uuid);
pub struct AgentId(String);              // role identity inside a swarm

pub enum RunMode { ModeA, ModeB, ModeC }

pub struct NanoAgent {
    pub id: NanoAgentId,
    pub version: SemVer,
    pub run_mode: RunMode,
    pub input_schema: JsonSchema,
    pub output_schema: JsonSchema,
    pub compliance_tags: ComplianceTags,
    pub pricing: PricingTier,
    pub publisher: PublisherRef,
    pub tier: PublisherTier,             // 0 / 1 / 2 / audit
}

pub struct SwarmRequest {
    pub request_id: Uuid,
    pub tenant_id: TenantId,
    pub caller_agent_id: Option<String>,
    pub operations: Vec<Operation>,
    pub max_parallelism: u32,
    pub timeout_ms: u64,
    pub idempotency_key: Option<String>,
    pub streaming: bool,
    pub audit_tags: Vec<String>,
    pub cost_cap_cents: Option<u64>,
    pub parent_envelope_trace_id: Option<Uuid>,
}

pub struct Operation {
    pub op_id: String,
    pub agent: NanoAgentRef,
    pub input: OperationInput,           // Literal | BlackboardRead | DependencyResult
    pub depends_on: Vec<String>,
    pub timeout_ms: Option<u64>,
    pub retry: RetryPolicy,
}
```

Plafonds canoniques (constants dans le SDK) :

- `MAX_CONTEXT_BUNDLE_BYTES = 262_144` (256 Ko) — taille max du blackboard d'un swarm
- `MAX_OPERATIONS_PER_SWARM = 256`
- `MAX_DAG_DEPTH = 16`
- `MAX_PARALLELISM = 64`

7.2 Catalog

[SHIPPED] — Crate : `efe-nano::catalog`

Le **registre des nano-agents disponibles** pour un tenant. Implémentations :

- `InMemoryCatalog` [SHIPPED] — pour dev/test, populated via `register_all_starter_pack`
- `PostgresCatalog` [STUB] — pour prod (migration v004-v007 prête mais non câblée)

API :

```
#[async_trait]
pub trait Catalog: Send + Sync {
    async fn get(&self, r: &NanoAgentRef, t: TenantId) -> Result<NanoAgent>;
    async fn list(&self, t: TenantId, filter: CatalogFilter) -> Result<Vec<NanoAgent>>;
    async fn register(&self, a: NanoAgent, t: TenantId) -> Result<()>;
    async fn deprecate(&self, r: &NanoAgentRef, t: TenantId) -> Result<()>;
}
```

Le catalog est tenant-aware : un tenant pharma peut voir un `ner_clinical_rare_diseases` qui n'existe pas pour un tenant fintech.

7.3 Modes d'exécution A / B / C

[SHIPPED]

Mode	Latence	Coût/call	Déterminisme	Implémentation
A Pure Rust	1–10 ms	~0	Mathématique (input -> output bit-identical, forever)	`PureRustRuntime` + `HandlerRegistry`
B Local ML	50–200 ms	~0.1 ¢	Seeded (avec seed fixé, output bit-identical)	`LocalModelRuntime` + `VmPool` (StubVmPool ou Firecracker) + `ModelLoader`
C LLM + JSON schema	500–2000 ms	1–5 ¢	Cache-replayable (mêmes input + cache TTL -> même output)	`LlmAugmentedRuntime` + `LlmClient` + `OutputValidator` + retry

Mode A — Pure Rust :

- Handlers sont des `Fn(input: serde_json::Value, ctx: &CallContext) -> Result<serde_json::Value>`
- Stockés dans `HandlerRegistry: HashMap<String, Arc<dyn HandlerFn>>`
- Exécutés directement dans le runtime, sans IO

Mode B — Local ML :

- Modèles chargés au démarrage par `ModelLoader` (5 builtins : NER clinical, NER legal, intent classifier, lang detect, embed text)
- Exécutés dans une VM Firecracker isolée (ou `StubVmPool` en dev)
- Le `VmPool` route la requête vers une VM idle, attend le résultat, recycle la VM

Mode C — LLM augmented :

- Génère un prompt depuis le schema d'entrée

- Appelle un LLM via `LlmClient` (Anthropic / OpenAI / local via efe-llm-router)
- Valide la sortie contre le JSON schema (`OutputValidator`)
- Si invalide, retry jusqu'à 3 fois avec feedback
- PII redaction en pré-traitement via `PiiRedactor`
- Provenance record écrit par `ProvenanceCollectingLlmClient` wrapper

7.4 Dispatcher et DAG

[SHIPPED] — Crate : `efe-nano::dispatcher`

Le **Dispatcher** reçoit un `SwarmRequest`, planifie un DAG d'exécution, et orchestre les nano-agents.

Algorithme :

1. Topological sort du DAG d'opérations
2. Pour chaque niveau (operations sans dépendance non résolue) :

a. Lance les opérations en parallèle (jusqu'à `max_parallelism`) b. Chaque opération récupère son input depuis l'environnement (literal, blackboard, dependency result) c. Appelle le runtime correspondant (A/B/C via `MultiModeRuntime`) d. Si succès, propage le résultat aux dépendants e. Si échec, applique `RetryPolicy` puis remonte l'erreur

1. À chaque appel, écrit dans le HMAC chain
2. Retourne un `SwarmResponse` agrégé

Anti-patterns :

- [N/A] Dépendance circulaire -> rejeté à la validation
- [N/A] DAG profondeur > 16 -> rejeté
- [N/A] Total parallélisme > 64 -> rejeté
- [N/A] Cost cap dépassé -> opération avortée, swarm en `PartialSuccess`

7.5 Blackboard et stigmergie

[SHIPPED] — Crate : `efe-blackboard`

C'est l'innovation centrale. Implémentation actuelle = primitive de stigmergie.

API :

```
pub struct Blackboard {
    fields: DashMap<String, BlackboardValue>,
    swarm_id: SwarmId,
    write_history: Mutex<Vec<BlackboardWrite>>,
    total_bytes: AtomicUsize,
    max_bytes: usize,
}

impl Blackboard {
    pub fn write(&self, ptr: &str, val: Value, agent: AgentId) -> Result<u64>;
    pub fn write_if(&self, ptr: &str, expected_v: u64, val: Value, agent: AgentId) -> Result<u64>;  // CAS
    pub fn read(&self, ptr: &str) -> Option<BlackboardValue>;
    pub fn snapshot(&self) -> BlackboardSnapshot;
    pub fn write_history(&self) -> Vec<BlackboardWrite>;
}

pub struct BlackboardValue {
    pub value: serde_json::Value,
    pub version: u64,                    // monotonic per pointer
    pub written_by: AgentId,
    pub written_at: SystemTime,
}
```

Concurrence :

- `DashMap` pour accès parallèle sharded par clé
- `write_if` (CAS) atomique sous shard guard (post-audit fix B-CAS)
- 50 writers concurrents -> tous succès, version finale entre 1 et 50 (write non-CAS)
- Audit log via `Mutex<Vec<BlackboardWrite>>` — lock courte durée, jamais d'`await` détenu

Adressage : JSON Pointer (RFC 6901). `/sources/0/level_2_summary`, `/intent/type`, etc.

Cap de taille : 256 Ko par défaut (= `MAX_CONTEXT_BUNDLE_BYTES`). Write rejeté si dépassement, sans mutation partielle.

[S&S] Extension stigmergique (S&S, additive)

Pour faire fonctionner S&S, on **étend** (sans modifier) le `BlackboardValue` :

```
pub struct BlackboardValue {
    pub value: serde_json::Value,
    pub version: u64,
    pub written_by: AgentId,
    pub written_at: SystemTime,

    // [S&S] S&S extension
    pub pheromone: f32,                   // 0..1, monotonic increase on useful read
    pub last_useful_ts: Option<SystemTime>,
    pub use_count: u32,
}
```

Et on ajoute un trait :

```
pub trait PheromoneOps {
    fn reinforce(&self, ptr: &str, delta: f32);    // appelé quand une capsule est citée
    fn decay(&self, lambda: f32, dt: Duration);    // cron: =1/24h
    fn ranked_snapshot(&self, top_k: usize) -> Vec<BlackboardValue>;  // ranked by pheromone
}
```

Sémantique :

- Quand L3 utilise une capsule pour construire le prompt cloud, `reinforce(ptr, 0.1)`
- Quand L4 cite une capsule dans la réponse, `reinforce(ptr, 0.5)`
- Quand L5 confirme une capsule (fact-check), `reinforce(ptr, 0.3)`
- Toutes les heures, un cron appelle `decay(0.95, 1h)` -> diminution exponentielle
- Les capsules à faible phéromone sont éligibles à éviction (TTL court)

7.6 Swarm Orchestrator

[SHIPPED] — Crate : `efe-swarm-orchestrator`

Différent du Dispatcher (qui exécute un DAG de NanoAgents) : ici on orchestre des **SwarmAgents** qui *lisent et écrivent dans un blackboard partagé*.

Trait :

```
#[async_trait]
pub trait SwarmAgent: Send + Sync {
    fn agent_id(&self) -> AgentId;
    async fn step(
        &self,
        bb: Arc<Blackboard>,
        ctx: &SwarmContext,
    ) -> Result<SwarmStepReport>;
}
```

Différence NanoAgent vs SwarmAgent :

	NanoAgent	SwarmAgent
Input	JSON literal	Blackboard read
Output	JSON literal	Blackboard write
Coupling	low (stateless)	medium (shared state)
Use case	Sous-tâche atomique	Coordination cyclique

Le pont : `efe-nano-swarm-adapter` (Sprint N23) fournit l'adaptateur bidirectionnel — un NanoAgent peut être exposé comme SwarmAgent et vice-versa.

7.7 Scheduler de fairness

[SHIPPED] — Module : `efe-nano::scheduler` (N15)

Composants :

- **Admission gate** : refuse les nouvelles requêtes si le système est surchargé (queue > seuil)
- **DRR** (Deficit Round Robin) : fair queuing entre tenants
- **Token bucket** par tenant : rate limit avec burst
- **Inflight tracker** : compte les opérations en cours par tenant
- **Metrics** : queue depth, latency budget consumed, fairness ratio

Garantie : un tenant "noisy neighbor" ne peut pas monopoliser plus que sa quote-part garantie.

7.8 Idempotency + cache

[SHIPPED] — Modules : `efe-nano::idempotency` + crate `efe-cache`

- **Idempotency** : clé optionnelle sur `SwarmRequest`. Si déjà traitée dans la fenêtre TTL (60 s par défaut), on retourne la réponse cachée *sans* réexécuter le DAG.
- **Cache embeddings** : Redis, clé = `BLAKE3(text + model)`, TTL configurable
- **Cache LLM responses** : Redis, clé = `BLAKE3(prompt + model + schema)`, TTL = 5 min par défaut
- **Quotas tenants** : Redis-backed counters via `efe-metering`

7.9 Provenance LLM

[SHIPPED] — Crate : `efe-llm-provenance` (Sprint N17)

Pour *chaque* invocation Mode C, on enregistre un `LlmProvenanceRecord` :

```
pub struct LlmProvenanceRecord {
    pub call_id: Uuid,
    pub model_id: String,                 // "claude-opus-4-7"
    pub model_version: String,            // exact version from response header
    pub provider: LlmProvider,            // Anthropic / OpenAI / Local
    pub prompt_hash: [u8; 32],
    pub response_hash: [u8; 32],
    pub system_prompt_hash: Option<[u8; 32]>,  // for cached prompts
    pub temperature: f32,
    pub max_tokens: u32,
    pub stop_sequences: Vec<String>,
    pub input_tokens: u32,
    pub output_tokens: u32,
    pub cache_tokens_read: u32,           // from prompt caching
    pub cache_tokens_created: u32,
    pub cost_cents: u64,
    pub started_at: SystemTime,
    pub finished_at: SystemTime,
    pub finish_reason: String,
    pub api_request_id: Option<String>,   // upstream request_id
}
```

Permet de **rejouer** précisément un appel LLM auprès du provider (avec le même API request_id) ou d'auditer une réponse 7 ans plus tard.

7.10 Publisher Identity + KYC

[SHIPPED] — Crate : `efe-publisher-identity` (Sprint N18)

Tier ladder :

Tier	KYC	Signature	Track record	Schema-freeze garanti	Audit cosign
0	Aucun (email seul)	Ed25519 sur body	0	Aucune	Non
1	Verified (Onfido)	Ed25519	90 jours incident-free	12 mois	Non
2	Verified + cooldown	Ed25519 + rotation	12 mois incident-free	36 mois	Non
Audit	Verified + KYB	Ed25519 + cosign Fulcio	24+ mois	7+ ans	**Oui**

Sanctions screening : OFAC + EU sanctions + UK financial sanctions, checked at registration and re-checked quarterly.

Revocation cascade : si un publisher est révoqué, tous les nano-agents qu'il a publiés passent en `deprecated` immédiatement, et les invocations futures retournent `PublisherRevokedError`.

7.11 Gold Test Gate

[SHIPPED] — Crate : `efe-gold-test-gate` (Sprint N19)

Système qui valide qu'une nouvelle version d'un nano-agent (ou un patch) **ne régresse pas** par rapport à un golden set figé.

Composants :

- `GoldSet` immutable, signé par opérateur, stocké en `chain_store` avec HMAC
- `GoldRunLedger` (HMAC-chained) qui enregistre chaque run et son verdict
- Runner hermétique en Firecracker (isolation totale)
- 3 stratégies d'assertion : `StrictEquality` (Mode A), `NumericTolerance` (Mode B), `SemanticSimilarity` (Mode C, via embedding distance)
- **30-day drift sweep** : tous les golden sets sont re-runnés mensuellement pour détecter les régressions silencieuses (ex. provider LLM qui change subtilement)
- **Emergency bypass** signé par opérateur en cas d'incident production

7.12 Multi-Region Replay

[SHIPPED] — Crate : `efe-multi-region` (Sprint N16)

Permet de rejouer une décision dans n'importe quelle région où la chaîne d'audit a été expédiée.

Composants :

- `ChainShipment` : delta-encoded chain export, signé par tenant key
- `JwtAuth` : tokens d'authentification cross-region avec audience scope
- `MultiRegionEngine` : coordinateur qui sélectionne la région la plus proche disposant de la chaîne

Use case : un régulateur français demande à rejouer une décision prise sur un tenant US 5 ans plus tard. Le système expédie la sous-chaîne pertinente, vérifie l'intégrité, rejoue, retourne le résultat — sans transférer aucune donnée brute non-pertinente.

7.13 Agentic Bridge (Claude)

[SHIPPED] — Crate : `efe-agentic-bridge` (Sprint N26 + N27)

Le pont entre les nano-agents EFE et un LLM externe (Claude, GPT-4) pour orchestration *tool-use*.

Composants :

- `bridge.rs` — interface Agent -> EFE swarm
- `claude.rs` — client Anthropic
- `claude_stream/` — SSE parser, state machine, finalize logic
- `tool_use.rs` — sérialisation des nano-agents en outils Claude
- `task.rs` — décomposition de la requête utilisateur en DAG
- `retry/` — backoff exponentiel + classification d'erreurs + loop executor
- `pricing.rs` — tracking coût per-call

Flow :

1. L'utilisateur envoie une requête textuelle
2. Le bridge envoie au LLM la liste des nano-agents disponibles comme `tools`
3. Le LLM répond avec une suite de `tool_use` (un DAG implicite)
4. Le bridge convertit en `SwarmRequest`, dispatche
5. Les résultats remontent au LLM dans le contexte
6. Le LLM produit la réponse finale
7. Tout est streamé via SSE

Retry policy :

- Erreurs 429, 5xx -> backoff exponentiel jusqu'à 3 tentatives
- Erreur d'auth (401) -> fail fast, alerter admin
- Erreur de cost cap -> abandon avec `PartialSuccess`

7.14 Gateway HTTP + MCP Server

[SHIPPED] — Crate : `efe-gateway` (Sprint N22) + service Python `mcp-server`

Gateway HTTP : 21 endpoints REST + SSE. Authentification par API key (header `Authorization: Bearer efe-pk-...`).

Endpoints principaux :

```
POST   /v1/swarms                  - Dispatch a swarm (200 OK with response, or 202 with stream URL)
GET    /v1/swarms/{id}/stream      - SSE stream for a running swarm
GET    /v1/swarms/{id}             - Get final result
DELETE /v1/swarms/{id}             - Cancel running swarm
GET    /v1/catalog                 - List available agents for tenant
GET    /v1/catalog/{agent_id}      - Detail of one agent
POST   /v1/admin/tenants           - Onboard tenant (admin scope)
GET    /v1/audit/{tenant_id}/chain - Stream audit chain (admin scope)
POST   /v1/audit/{tenant_id}/replay/{request_id} - Replay decision
GET    /v1/metering/{tenant_id}    - Usage + cost
... (publisher management, gold gate ops, multi-region ops)
```

MCP Server (Python, séparé) : expose 78 outils MCP pour que des agents externes (Claude Desktop, OpenAI Assistants, custom agents) puissent appeler EFE comme s'il s'agissait de fonctions natives.

Exemples d'outils MCP exposés :

- `efe_classify_intent(text: str) -> IntentClassification`
- `efe_extract_structured(text: str, schema: dict) -> dict`
- `efe_search_documents(query: str, top_k: int) -> List[Document]`
- `efe_fact_check(claim: str, sources: List[str]) -> CheckResult`
- ...

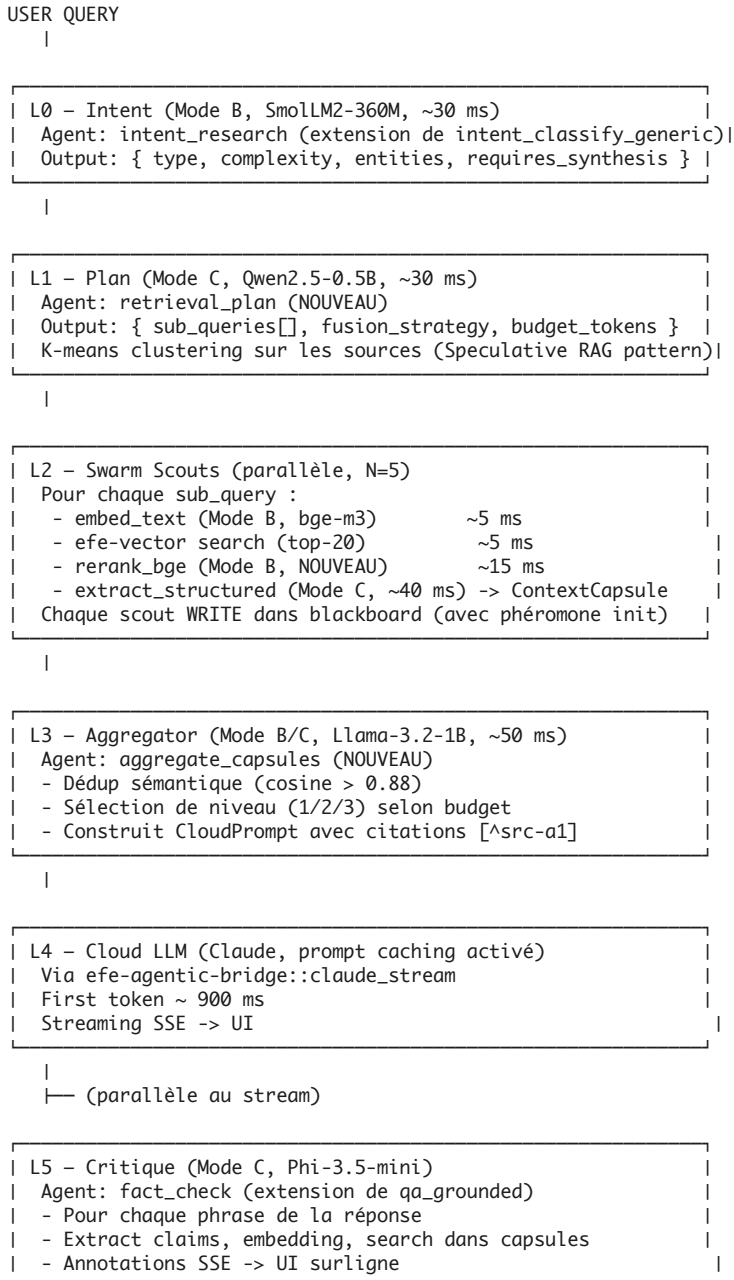
---

8. Application Scout & Synthesize

[S&S]

S&S est une **application** qui *consomme* la plateforme EFE pour exécuter le DAG de recherche documentaire à faible latence.

8.1 Vue logique



└──┘

8.2 Crates nouveaux à créer pour S&S ([S&S])

Tous **additifs** — aucun crate existant n'est modifié.

Crate	Rôle	Dépendances
`efe-nano-app-scout-synthesize`	DAG préconçu + types ContextCapsule + speculator	`efe-nano`, `efe-blackboard`, `efe-swarm-orchestrator`, `efe-agentic-bridge`, `efe-vector`, `efe-retrieval`, `efe-cache`
`efe-pheromone-extension` (optionnel)	Extension stigmergique du blackboard	`efe-blackboard`
`apps/scout-synthesize-ui` (frontend)	UI Svelte ou React consommant SSE	—

8.3 Types canoniques S&S

```
// in efe-nano-app-scout-synthesize::types

pub struct ContextCapsule {
    pub capsule_id: Uuid,                 // v7, sortable
    pub sub_query: String,
    pub source_id: String,
    pub source_meta: SourceMeta,          // title, date, url, page, ...
    pub relevance_score: f32,

    // 3 levels of compression
    pub level_1_oneline: String,          // ≤ 20 words
    pub level_2_summary: String,          // ≤ 80 words
    pub level_3_extracts: Vec<Extract>,

    // Stigmergy fields
    pub pheromone_weight: f32,
    pub last_seen_ts: SystemTime,

    // For L5 critique
    pub factual_claims: Vec<String>,
}

pub struct Extract {
    pub text: String,
    pub char_start: u32,
    pub char_end: u32,
    pub confidence: f32,
}

pub struct CloudPrompt {
    pub system: String,                   // cacheable Anthropic
    pub sources_block: String,            // markdown with [^src-N]
    pub question: String,
    pub expected_format: String,
    pub capsules_used: Vec<Uuid>,
    pub total_tokens_estimate: u32,
}

pub struct CritiqueAnnotation {
    pub sentence_start: u32,
    pub sentence_end: u32,
    pub claims: Vec<Claim>,
}

pub struct Claim {
    pub text: String,
    pub supported: bool,
    pub best_capsule_id: Option<Uuid>,
    pub similarity: f32,
    pub severity: CritiqueSeverity,
}
```

8.4 DAG préconçu (YAML)

```
# efe-nano-app-scout-synthesize/dags/scout_synthesize.yaml
name: scout_synthesize
version: 1.0.0
description: Document research with parallel scouts and parallel critique

operations:
  - op_id: l0_intent
    agent: { id: intent_research, version: "^1.0" }
    input: { type: literal, value: { query: "${request.query}" } }
    depends_on: []

  - op_id: l0_cache_check
    agent: { id: semantic_cache_lookup, version: "^1.0" }
    input: { type: dependency, source: l0_intent, path: /cache_key }
    depends_on: [l0_intent]
    # If cache hit, short-circuits the entire pipeline

  - op_id: l1_plan
    agent: { id: retrieval_plan, version: "^1.0" }
    input: { type: dependency, source: l0_intent }
    depends_on: [l0_intent]
    # Only runs if l0_cache_check returned miss

  - op_id: l2_scouts
    # Dynamic fan-out: produces N child operations based on l1_plan.sub_queries
    fan_out:
      from: l1_plan
      path: /sub_queries
      child_op_template:
        agent: { id: scout_step, version: "^1.0" }
        input: { type: from_fanout }
        depends_on: [l1_plan]
        write_blackboard: { ptr: "/sources/${index}", reinforce: 0.1 }

  - op_id: l3_aggregate
    agent: { id: aggregate_capsules, version: "^1.0" }
    input: { type: blackboard_snapshot, path: /sources }
    depends_on: [l2_scouts]

  - op_id: l4_cloud
    agent: { id: claude_synthesizer, version: "^1.0" }
    input: { type: dependency, source: l3_aggregate }
    depends_on: [l3_aggregate]
    streaming: true

  - op_id: l5_critique
    agent: { id: fact_check, version: "^1.0" }
    input:
      type: combined
      response_stream: { source: l4_cloud, mode: streaming_chunks }
      capsules: { type: blackboard_snapshot, path: /sources }
    depends_on: [l4_cloud]
    parallel_with: l4_cloud   # runs alongside, doesn't block

max_parallelism: 8
timeout_ms: 30000
audit_tags: [scout_synthesize, v1]
```

8.5 Spéculation pendant la frappe

[S&S] — Endpoint à ajouter dans `efe-gateway` (additif, sans toucher au reste) :

```
POST /v1/scout-synthesize/speculate
Headers: Authorization, X-Session-Id
Body: { query_partial: string }
Response: 204 No Content (fire-and-forget)

GET /v1/scout-synthesize/speculate/{session_id}/cache
Response: { cached_intent?, cached_plan?, ttl_ms }
```

Logique :

1. UI émet `query_change` toutes les 250 ms (debounced)
2. Gateway calcule un embedding bge-m3 rapide (~3 ms)
3. Si cos_sim > 0.85 avec dernière spéculation -> no-op

4 Sinon, lance L0 + L1 en arrière-plan, faible priorité (via scheduler)

5 Stocke le résultat dans Redis avec clé = embedding hash, TTL = 60 s

6 À soumission finale (`POST /v1/swarms` avec query complète), check cache : si hit, skip L0+L1 -> ~60 ms gagnés

8.6 Failover local

[S&S] — Extension de `efe-agentic-bridge` :

Si Claude TTFB > 1500 ms ou erreur 5xx persistante :

1 Avorter l'appel Claude (cancel HTTP)

2 Re-router le `CloudPrompt` vers `efe-llm-router` avec provider = local Llama-3.2-1B

3 Marquer la réponse avec badge `[local] réponse locale`

4 UI propose bouton "Régénérer avec Claude"

---

9. Architecture des données

9.1 Matrice de stockage

Donnée	Store	Persistence	Latence read	Volume estimé
Catalog nano-agents	Postgres	Permanent	~5 ms	< 100 K rows
Audit chain	Postgres (sharded by tenant)	7 j à 7 ans	~5 ms	10■–10^{11} rows/tenant
Merkle epoch roots	Postgres + Sigstore (audit tier)	Permanent	< 1 ms	~9 K rows/an/tenant
Tenant config	Postgres	Permanent	< 5 ms	< 10 K rows
Embeddings	Redis (LRU) + Postgres pgvector (cold)	TTL 24h hot, permanent cold	< 1 ms hot	10■–10^{12} vectors
LLM response cache	Redis	TTL 5 min	< 1 ms	~10■ entries hot
Blackboard (per-swarm)	In-memory (ephemeral)	Vie du swarm	< 1 ms	≤ 256 Ko/swarm
Vector documents (chunks)	efe-vector (HNSW + flat)	Permanent	< 5 ms	10■–10■ chunks
Knowledge graph	efe-kg	Permanent	~10 ms	10■–10■ edges
Quotas/metering	Redis counters + ClickHouse rollup	Real-time + 90j cold	< 1 ms	10■ events/jour
Sessions speculator	Redis	TTL 5 min	< 1 ms	< 10■ keys
Traces (OTLP)	Tempo	7 j	—	scale-dependent

9.2 Schémas Postgres clés

```
-- HMAC audit chain (sharded by tenant_id)
CREATE TABLE audit_chain_<tenant_uuid_short> (
    entry_id        UUID PRIMARY KEY,
    chain_seq       BIGINT NOT NULL,
    request_id      UUID NOT NULL,
    op_id           TEXT NOT NULL,
    agent_ref       TEXT NOT NULL,
    run_mode        SMALLINT NOT NULL,
    started_at      TIMESTAMPTZ NOT NULL,
    finished_at     TIMESTAMPTZ NOT NULL,
    duration_ms     INTEGER NOT NULL,
```

```
    input_hash      BYTEA NOT NULL,         -- 32 bytes
    output_hash     BYTEA NOT NULL,
    full_input      JSONB,                  -- nullable for tombstoned
    full_output     JSONB,
    status          SMALLINT NOT NULL,
    error_body      JSONB,
    prev_hash       BYTEA NOT NULL,
    entry_hmac      BYTEA NOT NULL,
    llm_provenance  JSONB,
    tags            TEXT[],
    UNIQUE (chain_seq)
);

CREATE INDEX idx_request ON audit_chain_xxx (request_id);
CREATE INDEX idx_time    ON audit_chain_xxx (started_at);
CREATE INDEX idx_agent   ON audit_chain_xxx (agent_ref);

-- Merkle epoch roots (one row per epoch per tenant)
CREATE TABLE merkle_epochs (
    tenant_id       UUID NOT NULL,
    epoch_seq       BIGINT NOT NULL,
    range_start_seq BIGINT NOT NULL,
    range_end_seq   BIGINT NOT NULL,
    merkle_root     BYTEA NOT NULL,         -- 32 bytes
    cosign_bundle   BYTEA,                  -- for audit tier
    computed_at     TIMESTAMPTZ NOT NULL,
    PRIMARY KEY (tenant_id, epoch_seq)
);
```

9.3 Chunking documentaire (S&S)

Pour l'application S&S, les documents sont chunkés en 3 niveaux dans `efe-vector` :

```
table chunks_section   (~ 2000 tokens)
  ├── chunk_id, doc_id, text, vector(1024), meta, pheromone, use_count
table chunks_paragraph (~ 300 tokens, overlap 50)
  ├── chunk_id, doc_id, section_id, text, vector(1024), meta, pheromone, use_count
table chunks_sentence  (~ 50 tokens)
  └── chunk_id, doc_id, paragraph_id, text, vector(1024), meta
```

Le retrieval initial frappe `chunks_paragraph` (sweet spot recall/precision). Pour la diversité, on remonte en `chunks_section`. Pour les citations littérales, on descend en `chunks_sentence`.

---

10. Plan de contrôle et streaming

10.1 Protocole SSE (Server-Sent Events)

Tous les endpoints `/stream` émettent des événements typés :

```
type StreamEvent =
  | { type: "ack",        request_id: string, trace_id: string }
  | { type: "progress",   layer: "L0"|"L1"|"L2"|"L3"|"L4"|"L5", status: string, t_ms: number }
  | { type: "intent",     payload: IntentResult }
  | { type: "plan",       payload: RetrievalPlan }
  | { type: "source",     payload: ContextCapsule }     // one per scout when ready
  | { type: "prompt",     tokens_used: number, cached_tokens: number }
  | { type: "token",      text: string }                // LLM streaming
  | { type: "critique",   payload: CritiqueAnnotation }
  | { type: "done",       stats: RequestStats, audit_entry_id: string }
  | { type: "error",      code: string, message: string, recoverable: boolean }
```

L'UI peut afficher chaque type d'événement comme un signal de progression distinct. C'est ce qui découple la latence perçue de la latence totale.

10.2 Cycle de vie d'une requête S&S (timeline cible)

```
T+0 ms    POST /v1/swarms (query)
T+5 ms    Gateway -> dispatcher
T+15 ms   ack émis (request_id, trace_id)
T+30 ms   L0 done -> "intent" émis
T+60 ms   L1 done -> "plan" émis (UI affiche sub-questions)
T+150 ms  L2 scouts done (5 capsules) -> "source" × 5 émis
          *** USEFUL PIXEL : UI montre 5 sources cliquables ***
T+200 ms  L3 done -> "prompt" émis (cloud prompt construit)
T+250 ms  Claude API call commence
T+900 ms  Claude first token -> "token" stream commence
T+1000 ms L5 commence (parallèle au stream)
T+2500 ms Claude done streaming
T+2700 ms L5 done -> "critique" annotations émises
T+2750 ms "done" émis (stats finales, audit_entry_id)
```

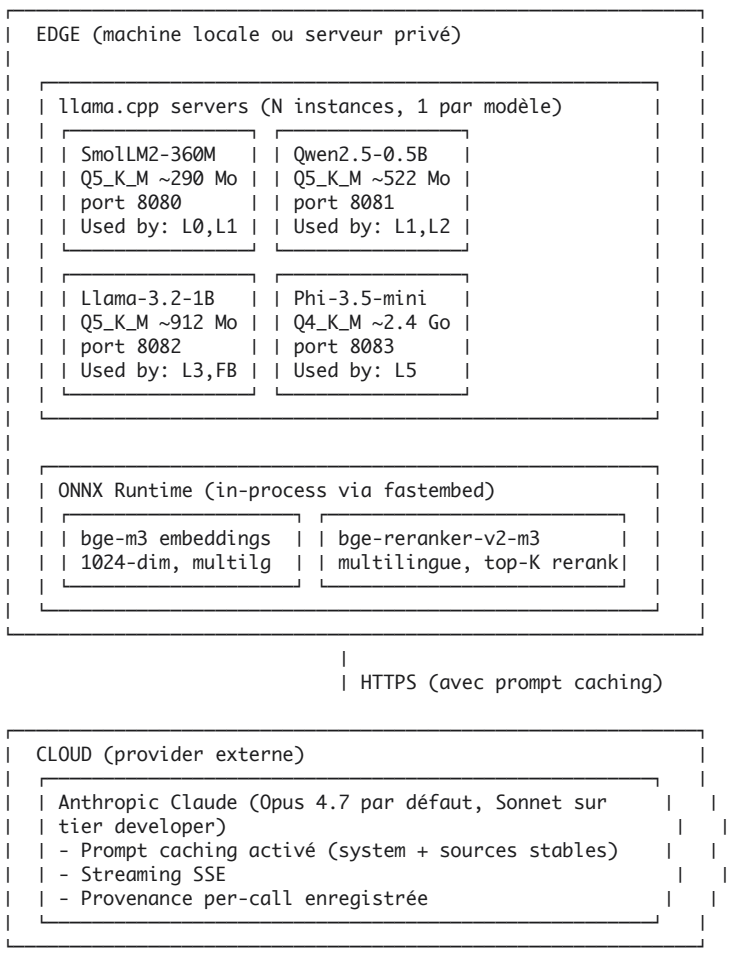
10.3 Annulation et backpressure

- Chaque requête a un `request_id` UUIDv7
- Cancellation propagée via `tokio::CancellationToken`
- Si UI ferme le SSE, le gateway annule l'opération via `DELETE /v1/swarms/{id}`
- Backpressure naturelle : si le client SSE consomme lentement, le gateway ralentit son `yield`, qui ralentit la lecture du stream Claude

---

11. Plan d'inférence

11.1 Topologie des modèles (edge/cloud hybride)



11.2 Empreinte mémoire totale (edge complet)

Composant	RAM	Disque
SmolLM2-360M Q5	~290 Mo	290 Mo
Qwen2.5-0.5B Q5	~522 Mo	522 Mo
Llama-3.2-1B Q5	~912 Mo	912 Mo
Phi-3.5-mini Q4	~2.4 Go	2.4 Go
bge-m3 ONNX	~1.2 Go	2.3 Go
bge-reranker-v2-m3 ONNX	~1.2 Go	1.1 Go
efe-server (Rust)	~500 Mo	50 Mo
efe-gateway (Rust)	~200 Mo	30 Mo
Postgres	~2 Go	scale-dep
Redis	~1 Go	—
Total minimal	**~10 Go**	**~7.5 Go**

Tient sur un **Mac M2/M3 16 Go** ou un **serveur Linux 32 Go** avec marge.

11.3 Routing Mode C (LLM augmenté)

`efe-llm-router` route chaque appel Mode C selon :

- Le `provider` configuré pour le tenant
- La charge actuelle (circuit breaker par provider)
- Le coût marginal (préfère provider le moins cher si qualité égale)
- La région (data residency)

Fallback automatique en cas de circuit ouvert.

---

12. Sécurité, isolation, conformité

12.1 Isolation par tenant

Chaque tenant a :

- **Une clé HMAC** dérivée via HKDF d'une racine en KMS
- **Une chaîne d'audit** dédiée (table Postgres ou shard)
- **Des quotas Redis** distincts
- **Des keys Redis namespacées** (`tenant:{uuid}:*`)
- **Une affinity de scheduling** pour réduire le bruit cross-tenant
- **Un set de nano-agents visibles** (catalog tenant-aware)

12.2 Isolation par invocation (Firecracker)

Chaque invocation Mode B et Mode C tourne dans une **microVM Firecracker** :

- Kernel mince (minikernel sans drivers superflus)
- Rootfs read-only (sauf `/tmp` tmpfs)
- Pas de réseau sauf vsock vers contrôleur

- `agent-core.py` est le seul process autorisé à s'exécuter
- Pool de VMs pré-bootées (~200 ms boot, ~50 ms reuse)
- VM détruite après usage (anti-state-leak)

12.3 PII redaction

Crate : `efe-pii`. Appelé en Mode C avant l'envoi au LLM externe.

- Détection regex + heuristiques (emails, phones, SSN, IBAN, etc.)
- Politique par tenant (strict / balanced / permissive)
- Circuit breaker : si > 5 % de redactions sur 1 min -> escalade humaine
- Leak detection : check la réponse LLM pour absence de PII redactée

12.4 KYC + sanctions

Onfido (KYC individuals), Sanctions.io (OFAC + EU + UK). Re-screening trimestriel automatique. Revocation cascade si match.

12.5 Cosign signing (audit tier uniquement)

Pour le tier audit, chaque Merkle epoch root est signé via Sigstore Fulcio + ancré dans Rekor (transparency log). Un régulateur peut vérifier la signature 7 ans plus tard contre l'historique public de Sigstore.

12.6 Conformité explicite

Régulation	Mécanisme EFE
FINMA (suisse)	HMAC chain + cosign + replay E2E
DORA (UE finance)	Multi-region replay + audit retention 5+ ans
AI Act (UE)	Provenance LLM + transparency log + risk classification per agent
FDA GxP (pharma)	Gold test gate + drift sweep + signed binary releases
HIPAA (US santé)	PII redaction + tenant isolation + BAA-compatible audit
GDPR (UE)	Tombstone mechanism + right-to-erasure cascade

---

13. Observabilité

13.1 Tracing

Crate : `efe-telemetry`. Tous les composants émettent des spans OpenTelemetry avec `trace_id` propagé via `traceparent` headers.

Span hierarchy d'une requête S&S :

```
trace_id: 01H...
└─ span: request                                  t=0,    d=2.7s
   ├─ span: gateway.auth                          t=0,    d=2 ms
   ├─ span: gateway.dispatch                      t=2,    d=8 ms
   ├─ span: dispatcher.L0.intent                  t=10,   d=20 ms
   │  └─ span: llama.infer model=smollm2          t=12,   d=16 ms
   ├─ span: dispatcher.L1.plan                    t=30,   d=28 ms
   │  └─ span: llama.infer model=qwen2.5          t=32,   d=24 ms
   ├─ span: dispatcher.L2.swarm                   t=58,   d=92 ms
   │  ├─ span: scout subq=0                       t=58,   d=88 ms
   │  │  ├─ span: embed                           d=4 ms
```

```
   |  |  ├─ span: vector.search                 d=5 ms
   |  |  ├─ span: rerank                        d=16 ms
   |  |  └─ span: capsule.extract               d=60 ms
   |  ├─ span: scout subq=1                     ...
   |  └─ ...
   ├─ span: dispatcher.L3.aggregate             t=150,  d=48 ms
   ├─ span: dispatcher.L4.cloud                 t=198,  d=2.3s
   |  ├─ event: first_token                     t=900
   |  ├─ span: agentic-bridge.claude.stream     d=2.1s
   |  └─ span: provenance.write                 d=4 ms
   └─ span: dispatcher.L5.critique              t=900,  d=1.8s (parallel with L4)

trace_id sauvegardé dans audit entries pour cross-référence.
```

13.2 Métriques (Prometheus, taxonomie EFE)

```
# Latence par couche
efe_nano_latency_ms{layer="L0|L1|...", mode="A|B|C", tenant_id}    histogram

# Taux d'échec et de timeout
efe_nano_failure_rate{layer, mode}                                  counter
efe_nano_timeout_rate{layer, mode}                                  counter

# Cache hits
efe_cache_hit_rate{cache="L0|L2|llm|embedding|prompt"}              gauge

# Escalation vers cloud
efe_swarm_cloud_escalation_rate                                     gauge
efe_swarm_local_fallback_rate                                       gauge

# Audit chain
efe_audit_chain_length{tenant_id}                                   gauge
efe_audit_chain_writes_per_sec{tenant_id}                           counter
efe_merkle_epoch_lag_ms                                             gauge

# Stigmergy (S&S)
efe_blackboard_size_bytes{swarm_id}                                 gauge
efe_pheromone_distribution_p50{ptr_prefix}                          gauge
efe_blackboard_writes_total{agent_id}                               counter

# Infrastructure
efe_llama_slot_queue_depth{model}                                   gauge
efe_llama_slot_utilization{model}                                   gauge
efe_firecracker_vm_pool_free                                        gauge

# Coût
efe_cloud_tokens_input{tenant_id, provider}                         counter
efe_cloud_tokens_cached_read{tenant_id, provider}                   counter
efe_cloud_cost_cents{tenant_id, provider}                           counter

# Qualité (depuis L5)
efe_unsourced_claims_per_response                                   histogram
efe_critique_severity{severity="INFO|WARN|ERROR"}                   counter

# SLOs
efe_slo_latency_budget_consumed{slo_name}                           gauge
efe_slo_error_budget_consumed{slo_name}                             gauge
```

13.3 SLOs (cf. `SLO.md`)

SLO	Objectif	Fenêtre
Latence Mode A p99	< 50 ms	30 jours
Latence Mode B p99	< 500 ms	30 jours
Latence Mode C p99	< 5 s	30 jours
Latence S&S TTUPE p99	< 500 ms	30 jours
Audit chain coherence	100 % (zero broken chains)	toujours
Disponibilité gateway	99.9 %	28 jours roulants
Erreur rate global	< 0.5 %	28 jours roulants

14. Tenancy et configuration

14.1 Modèle de tenancy

Trois niveaux :

- **Cluster** : un déploiement EFE (1+ régions)
- **Tenant** : une entité légale (entreprise, équipe)
- **Project** (optionnel) : sous-division d'un tenant pour isoler des workloads

Chaque niveau a sa configuration :

```
# config/cluster.yaml
cluster:
  region: eu-west-1
  meta_swarm_enabled: false
  audit_kms_provider: vault
  audit_kms_path: secret/efe/audit

tenant:
  default_tier: developer
  default_quotas:
    requests_per_minute: 60
    requests_per_day: 10000
    cost_cap_usd_per_day: 100

project: {}  # inherits from tenant
```

14.2 Configuration hiérarchique

```
cluster.yaml -> tenant.yaml -> project.yaml -> request headers
```

Le runtime résout la config via merge (project overrides tenant overrides cluster).

14.3 Onboarding d'un tenant

CLI : `efe-admin tenant create`. Flow :

1. Vérification KYC (Onfido)
2. Création de la clé HMAC (KMS)
3. Création du shard audit Postgres
4. Setup Redis namespace
5. Setup quotas + tier
6. Génération API key
7. First-time setup wizard pour le catalog (selection des nano-agents activés)

15. Évaluation et boucles qualité

15.1 Gold Test Gate (rappel)

Cf. §7.11. Cœur de la qualité.

15.2 Drift monitoring

Crate : `efe-curation` + workflow `.github/workflows/al-drift-monitor.yml`

Tous les 30 jours, chaque tenant peut re-runner son golden set pour détecter une régression silencieuse (typiquement parce qu'un provider LLM upstream a évolué).

15.3 Active learning (crate `efe-active-learning`)

Pour les nano-agents Mode B/C, boucle continue :

1. Production échantillonne les opérations à forte incertitude
2. Un LLM oracle (Claude Opus) annote
3. Annotations vont dans un gold dataset
4. Centroid retune périodique (Mode B)
5. Eval + canary
6. Promotion si métriques OK

15.4 Distillation ([S&S] S&S extension)

Pour S&S, ajout d'une boucle hebdomadaire :

1. Capture des (input_L4_prompt, response_Claude, score_Quality) en production
2. Curation : garde les cas où Claude a corrigé/réécrit substantiellement
3. LoRA fine-tune des nanos Mode B/C avec unsloth
4. A/B test offline sur golden set
5. Promote si delta_quality > +1 % et delta_latency < +10 %
6. Hot-swap LoRA adapter sans restart

---

16. Cycle de vie d'une requête (E2E)

Voici le **walkthrough complet** d'une requête S&S réelle.

16.1 Phase 0 — Authentification et admission

1. UI ouvre une session, obtient API key
2. UI envoie `POST /v1/swarms` avec header `Authorization: Bearer efe-pk-...`
3. Gateway valide la clé (cache Redis)
4. Gateway vérifie quota tenant (token bucket Redis)
5. Gateway vérifie tier (developer ≤ 1 cent/call, etc.)
6. Si tout OK, génère `request_id` (UUIDv7), `trace_id`, et émet `ack` SSE

16.2 Phase 1 — Spéculation (déjà partiellement préchauffée)

- Pendant que l'utilisateur tapait, le gateway a déjà spéculativement lancé L0+L1
- Le cache Redis a une entrée `speculation:{embedding_hash}` valide
- Le dispatcher saute L0+L1 en utilisant le cache -> gain ~60 ms

16.3 Phase 2 — Dispatch du DAG

- Dispatcher charge le DAG `scout_synthesize.yaml`

- Topological sort des opérations
- Création du `Blackboard` pour ce `request_id`
- Création du `CallContext` avec `AuditWriter` HMAC chain pointant vers le tenant
- Démarrage des spans OTLP

16.4 Phase 3 — Exécution parallèle L2

- 5 sub-queries -> 5 scouts en parallèle
- Chaque scout :

- Embed via bge-m3 (~5 ms) - Search efe-vector top-20 (~5 ms) - Rerank top-20 -> top-5 (~15 ms) - Extract capsule via Qwen2.5-0.5B avec JSON schema (~40 ms)

- Chaque scout WRITE dans `blackboard` à `/sources/{i}` avec phéromone initial = 0.1
- SSE émet `source` events au fur et à mesure

16.5 Phase 4 — Aggregation L3

- L3 lit `blackboard.snapshot()`, filtre `/sources/*`
- Dédup sémantique (cosine > 0.88)
- Build `CloudPrompt` avec sources rangées par diversité + score
- SSE émet `prompt` avec compte de tokens
- Marque les capsules utilisées dans blackboard pour reinforcement

16.6 Phase 5 — Cloud LLM L4

- `efe-agentic-bridge` envoie à Claude avec `prompt_caching: true`
- Le system prompt + sources_block sont marqués cacheable
- Premier token revient ~900 ms (avec cache hit, ~600 ms)
- Stream SSE direct vers UI
- Pour chaque capsule citée dans la réponse, `blackboard.reinforce(ptr, 0.5)`
- Provenance record écrit en parallèle

16.7 Phase 6 — Critique L5 (parallèle au stream)

- L5 reçoit les tokens au fur et à mesure
- Buffer par phrase (heuristique de fin de phrase)
- Pour chaque phrase, extract claims via Phi-3.5-mini
- Pour chaque claim, embed + search dans `/sources` du blackboard
- Émet `critique` SSE annotations avec offsets

16.8 Phase 7 — Finalisation

- Stream Claude terminé
- L5 traite le dernier chunk
- Audit chain : flush des entries en attente
- Stats finales calculées (latence par couche, coût total, tokens utilisés)
- `done` SSE émis avec `audit_entry_id`
- Phéromones consolidées sur blackboard

- Blackboard snapshot stocké pour replay éventuel (24 h)
- Connection SSE fermée

16.9 Phase 8 — Audit & metering (asynchrone)

- HMAC chain entries committées dans Postgres
- Si epoch boundary atteinte, calcul du nouveau Merkle root
- Metering counters incrémentés (Redis -> ClickHouse rollup)
- Pour tier audit, cosign signature programmée
- Distill log : capture (input, output, claude_correction) si delta qualité

---

17. Index des crates

17.1 Par couche

Couche 1 — Inférence (5) : `efe-core`, `efe-kernels`, `efe-ffi`, `efe-server`, `efe-telemetry`

Couche 2 — Meta-Swarm A1 (3) : `efe-meta-swarm`, `efe-meta-swarm-proto`, `efe-meta-swarm-server`

Couche 3 — Audit (modules dans `efe-nano`) : `audit::hmac_chain`, `audit::chain_store`, `audit::hkdf`, `audit::verify`, `audit::merkle`, `replay`

Couche 4 — Nano Agent Platform (14) :

- Public SDK : `efe-nano-sdk`
- Runtime : `efe-nano`
- Guest daemon : `efe-nano-inference-daemon`
- 6 SwarmAgents réf. : `efe-nano-preanalysis`
- Adapter : `efe-nano-swarm-adapter`
- Multi-region : `efe-multi-region`
- Provenance : `efe-llm-provenance`
- Publisher identity : `efe-publisher-identity`
- Gold gate : `efe-gold-test-gate`
- Blackboard : `efe-blackboard`
- Swarm orchestrator : `efe-swarm-orchestrator`
- Agentic bridge : `efe-agentic-bridge`
- Gateway HTTP : `efe-gateway`
- Compliance wiring : `efe-compliance-wiring`

Hardening prod (18) : `efe-prompts`, `efe-llm-router`, `efe-admin`, `efe-saga`, `efe-curation`, `efe-active-learning`, `efe-agent-runner`, `efe-cache`, `efe-events`, `efe-pii`, `efe-multimodal`, `efe-region`, `efe-metering`, `efe-billing`, `efe-vector`, `efe-retrieval`, `efe-kg`, `efe-graphrag`

Tooling (3) : `efe-bench`, `efe-backup`, `efe-cluster-provisioner`

Tools workspace (2) : `tools/loadgen`, `bench/harness`, `smoke/runner`, `slos/rollup`

[S&S] Extension S&S (à venir, 1) : `efe-nano-app-scout-synthesize`

17.2 Par dépendance (top 5 fan-out)

Crate	Dépend de
`efe-nano-app-scout-synthesize` [S&S]	efe-nano, efe-blackboard, efe-swarm-orchestrator, efe-agentic-bridge, efe-vector, efe-retrieval, efe-cache
`efe-compliance-wiring`	efe-nano-swarm-adapter, efe-publisher-identity, efe-gold-test-gate
`efe-nano-swarm-adapter`	efe-nano, efe-swarm-orchestrator, efe-publisher-identity, efe-gold-test-gate, efe-llm-provenance
`efe-swarm-orchestrator`	efe-nano-sdk, efe-blackboard
`efe-nano`	efe-nano-sdk, efe-cache, efe-pii, efe-llm-router

17.3 Crates publics (MIT) vs commerciaux (UNLICENSED)

Public MIT : `efe-nano-sdk` uniquement. **Commercial UNLICENSED** : tous les autres.

Cette asymétrie est délibérée : on expose un SDK pour que des publishers tiers construisent des nano-agents, sans donner accès au runtime.

---

18. Glossaire

Terme	Définition
EFE	*Ephemeral Fluid Swarm*. La plateforme.
S&S	*Scout & Synthesize*. L'application de recherche documentaire.
A1	Aggregation 1. La couche meta-swarm de fédération.
Mode A/B/C	Modes d'exécution d'un nano-agent : pure Rust / ML local / LLM + JSON schema.
Nano-agent	Unité atomique d'exécution dans EFE (Mode A/B/C). Pas nécessairement un LLM.
SLM-nano-agent	Sous-ensemble Mode B/C qui *utilise* un small language model.
SwarmAgent	Agent qui communique via blackboard (différent de NanoAgent atomique).
Blackboard	Mémoire partagée concurrente, JSON-pointer addressable, avec versioning et phéromones.
Stigmergie	Coordination indirecte via modifications de l'environnement partagé (inspiré des fourmis).
Phéromone	Valeur flottante 0..1 sur une cellule blackboard, augmentée à chaque usage utile, décroissance exponentielle.
Capsule	`ContextCapsule` : extrait sourcé avec 3 niveaux de compression, l'unité de transport en S&S.
HMAC chain	Chaîne d'audit cryptographique par tenant, chaque entrée intègre le hash de la précédente.
Merkle epoch	Période (1 h par défaut) au bout de laquelle on calcule la racine Merkle de la chaîne.
Beacon	(Symphony) Message broadcast pour service discovery + capability matching.
Drafter/Verifier	(Speculative RAG) Pattern où un petit modèle propose K candidats que le gros vérifie.
Speculator	Sous-système qui lance L0+L1 pendant la frappe (pre-warming).
Tier	Niveau de garantie d'un publisher (0/1/2/audit) ou de rétention d'un tenant.

Terme	Définition
Cosign	Système de signature Sigstore avec ancrage transparency log (Rekor).
Firecracker	MicroVM Amazon basée sur KVM, isolation kernel-level, boot < 200 ms.
DAG	*Directed Acyclic Graph* d'opérations dans un swarm.
TTUPE	*Time To Useful Pixel Emitted* : latence ressentie (premier événement utile à l'UI).
TTFT	*Time To First Token* : premier token du gros LLM.
Provenance	Enregistrement forensique per-call (LLM uniquement, Mode C).
Gold gate	Système de validation contre un golden set immuable.
Replay	Re-exécution d'une décision passée pour vérification par régulateur.

En une phrase

Le système est une plateforme Rust (EFE) de 47 crates qui *audit-grade-ifie par construction* tout flux d'agent IA via des nano-agents Mode A/B/C orchestrés en DAG, coordonnés par un blackboard concurrent stigmergique, exécutés derrière un gateway HTTP+SSE et un MCP server à 78 outils, le tout transverseé par une chaîne HMAC + Merkle epochs réplayable sur 7 ans dans toute région, fédéré inter-instances par un protocole meta-swarm A1 *préservant la souveraineté des données* — et *Scout & Synthesize* est l'application qui consomme cette plateforme pour exécuter un DAG préconçu (L0 intent -> L1 plan -> L2 scouts parallèles avec phéromones -> L3 aggregation -> L4 Claude streaming -> L5 critique parallèle) avec spéculation pendant la frappe, failover local, et latence perçue inférieure à 250 ms.